

05-实施指南-P2-KV感知推理网关

P2 实施指南：KV-Cache 感知推理网关（代号 KVRouter） — v3 重构版

2026-10-05 v3 重构（导师与学员讨论定稿）：v1/v2 的”功能列表式自研网关”定位作废——2026 年外部生态已将其大部分功能产品化。本文以”**独立复现 + 量化测量 + 前沿改进回馈上游**”重构，**取代 03-项目结构定稿.md §2 的 P2 增量栈定义**（03 保留为 v2 历史骨架）。配套：07-开源贡献指南.md（本项目与其阶段 3「自产需求 PR」合流）。目标仍是”照着能开工”。

1. 项目介绍

电梯陈述：独立复现 vLLM 前缀路由的两种信息源机制（引擎 KV 事件的”精准模式” vs 网关路由历史的”估算模式”），用同一负载台与部署起来的真实 AlBrix 三方对照量化；在此基础上定位并攻研一个上游尚未解决好的前沿问题——**弹性伸缩期的缓存冷启动协同**，改进经数据验证后回馈 AlBrix。

三重定位，缺一不可：

- 复现者：**两种路由信息源都亲手实现。厂商只发布结论（“精准更准”），不发布”多准、在什么负载形态下差距消失”的曲线——你发布曲线。
- 测量者：**自己的实现必须与**真实部署的 AlBrix** 上同一套负载对照。这是 P2 版的”JuiceFS 上秤”：没有对照组的一切性能声明都是自说自话。
- 前沿贡献者：**终局交付物不是”我仓库里有个功能”，而是”**我给 AlBrix 的 issue 里有测量数据，PR 里有被接受的修复**”。

明确不是什么：不是”做一个比 AlBrix 更好的路由器”（功能赛道必输）；不重造 Envoy 数据面；不做引擎内核改动。

2. 问题与动机（含 2026-10 生态现勘）

2.1 外部事实（2026-10-05 核实，构成立项前提）

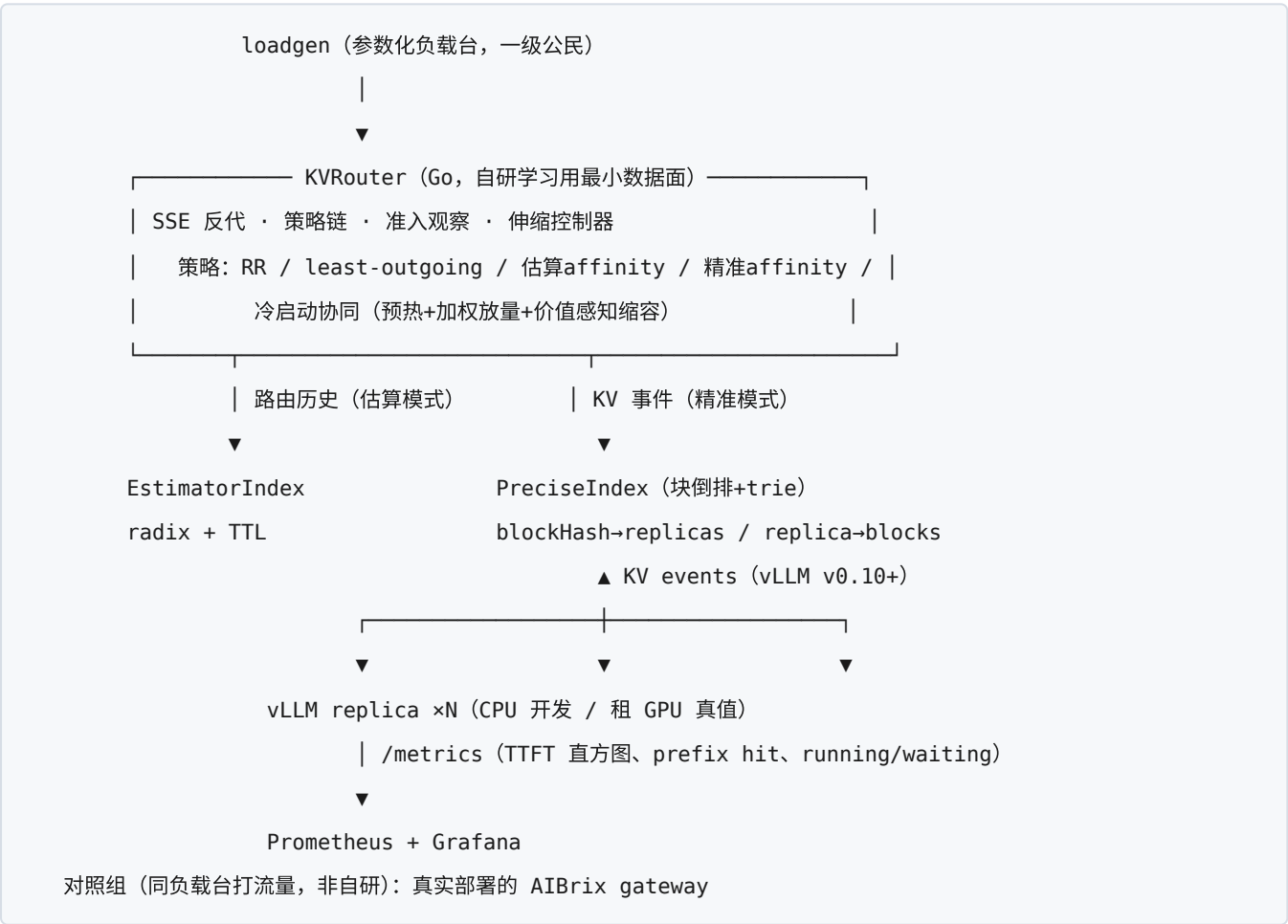
- vLLM v0.10+ 已支持 KV Cache 事件上报：**引擎主动把缓存块分布以事件消息推给网关。旧版设计中”vLLM 不会告诉你它缓存了什么，信息源只能是路由历史”的前提**已作废**。
- 两种模式已有行业命名：**阿里云 ACK 文档明确区分**精准模式**（消费引擎块事件）与**估算模式**（网关靠路由历史推断，与引擎解耦）。旧版 KVRetry = 较弱的估算模式。
- AlBrix (vllm-project) 已产品化：**v0.4（2025-08）KVCache Connector + KV Event Subscription；v0.7（2026-06）可加权混合软评分路由、高可用网关（Redis 状态同步）、请求优先级/限流、KV offload、P/D 分离（SHFS/NIXL/Mooncake 连接器），三个月合入 242 PR。其前缀索引为 block-based + recursive hash（约 256 token/block）+ trie。

- **llm-d** (Red Hat/Google/IBM/NVIDIA 发起, 2026-03 CNCF sandbox) 同样实现事件路由 + 全局块索引 + 分层 offload; **vLLM Production Stack** 早有 prefix-aware / KV-aware 路由教程。
- 含义: 前缀路由本身已是 commodity; “自研 radix + TTL” 不再是差异化, 而是每个合格候选人都该理解的基线机制。

2.2 仍然成立的问题（本项目的缝）

1. **伸缩期缓存协同没人做好**: 新副本 KV 全冷, 前缀感知路由天然不导流量给它 → 热身极慢甚至饿死; 扩容先杀谁、前缀损失多少、如何 drain, 公开资料基本空白。软评分中 resident KV 权重可能加剧冷副本饿死。
2. **精准 vs 估算的差距没有公开曲线**: 差距随” 共享前缀比例 × 缓存欠配程度” 如何变化? 什么负载下不值得为精准模式付引擎耦合?
3. **流式故障语义薄弱**: streaming 请求非幂等, 副本中途崩溃后能否安全重试, 文档与实现普遍含糊。

3. 架构总览



4. 阶段 0：现勘（与使用阶段合并，2–3 周，闸门制）

这是整个项目的地基，也是开源贡献指南阶段 0 的 P2 侧交付物。

动作清单：

- vLLM CPU 后端多副本跑通 (Qwen2.5-0.5B-Instruct)，通读启动参数中 KV 池、prefix caching、KV 事件上报相关项；
- AlBrix **quickstart (helm)** 部署成功即可，不要求本地编译全家桶；Production Stack helm 跑通 prefix-aware tutorial；
- 精读 AlBrix router 源码中：软评分、prefix index、KV event 消费、伸缩相关包；精读 **10 个**最近 merged PR (强制结构化笔记：diff 形状 / 解决什么问题 / reviewer 关注点 / 测试怎么写)；
- 产出 **《P2 对手现勘报告》**：部署命令与版本锁定 (镜像 digest)、各系统 feature 矩阵、复现的官方 benchmark 数据、带证据的缺口排序表。

头号技术风险 (现勘第一周必须关闭，不许带假设开工)：

vLLM CPU 后端是否上报与 GPU 后端相同 schema 的 KV 事件？ - 支持 → 精准模式全程本地开发； - 不支持 → 精准模式在 CPU 期只能用”符合锁定 schema 的 mock 事件流”开发逻辑，所有性能真值靠租 GPU，预算与排期按此重算。

缺口闸门：核心 2 选哪个缺口，由现勘证据决定，不由今天许愿。

候选缺口	判定它成立需要看到的证据	若被上游解决
A (默认押注) 伸缩期缓存冷启动：冷副本饿死、热身慢、缩容损失无度量	在 AlBrix 上制造扩容事件，新副本 5 分钟内流量份额/命中率仍显著低；官方无预热/放量机制	转 B 或 C
B 精准 vs 估算差距曲线	官方无公开二维对比 (共享比例 × 欠配度)	仍可做 (复现价值独立存在)，但不升为核心 2
C 流式故障安全重试	mid-stream kill 副本出现错误重放/挂死，issue 区无人认领	A 被解决时升为核心 2

5. 基座设计

5.1 引擎摸底与指标层

- CPU 模式：kind 3 副本 + 本机 2 副本；读懂 `vLLM_CPU_KVCACHE_SPACE` 等 KV 池参数并做笔记；
- MetricsView 抽象先行**：vLLM /metrics 系列名随版本漂移 (TTFT/TPOT 直方图、prefix cache hit/queries、num_requests_running/waiting、cache usage)，业务代码只依赖抽象接口；针对锁定版本写一份字段映射 + 缺字段降级；metric 名散落业务代码 = 监控系统第一课失败。
- 所有指标带 mode (rr/estimator/precise/aibrix)、replica、experiment_id 标签；实验配置 (模型、vLLM 版本、镜像 digest、负载参数) 随报告一起落盘。

5.2 Go 反代基座 (自研数据面的全部范围，难点在流式)

SSE 流式转发五坑清单 (每条实测后写进博客)：

- `httputil.ReverseProxy` 默认缓冲——`FlushInterval = -1` 才逐 token 下发；
- 客户端断开必须 cancel 上游 context，否则上游继续烧算力生成无人消费的 token；

- 3. SSE 禁压缩：Accept-Encoding 处理不当，gzip 缓冲把流式变成”一次性爆发”；
 - 4. idle timeout 随每个事件重置，而非请求总时长；
 - 5. TTFT 定义 = 请求发出到第一个内容 SSE 事件（data: 非心跳帧），不是首字节、不是 header。
- 其余：读写超时分离（ReadHeaderTimeout 有值、响应流无总超时）；每副本连接池 + /health 探针 + 连续失败摘除；X-Request-Id 全链路透传（核心 2/增量池复用）。

5.3 负载台（一级公民，所有旗舰图的唯一出口）

参数化场景生成器，参数固定为实验配置文件，保证三种自研策略与 AIBrix 对照组吃完全相同的流量：

维度	参数
共享前缀	共享比例 $f \in \{0, 0.25, 0.5, 0.8\}$ ；system prompt 池 $M=100$, Zipf(1.2) 热度
对话形态	单轮短问 / 多轮（轮数几何分布，均值 4） / 长文档 QA（2KB 固定前缀+不同问题）
到达过程	泊松 λ 扫点；突增实验： $3 \times \lambda$ 持续 60s
token 形状	输入/输出长度分布参数化（均值/尾部），以锁定 tokenizer 实际切分为准
缓存压力	调 KV 池大小制造 充裕/临界/欠配 三档
副本 churn	脚本按时间表 scale +1/-1，供事件研究

输出：TTFT/TBT 直方图、吞吐、各副本 inflight 与 KV usage、前缀命中率（引擎自报）、新副本流量份额。
陷阱：客户端本身饱和时数据作废——generator CPU/连接数常驻监控，结论只在”服务端饱和、客户端不满载” 区间成立。

6. 核心 1：双模式 KV 索引（替换旧增量 1）

6.1 估算模式（亲手复现一条被淘汰的路线，不是凑数）

- 前缀键：messages 按消息边界取哈希序列 $[h(\text{system}), h(\text{user1}), h(\text{asst1}), \dots]$ ，radix 树键 = 哈希序列前缀；
- 数据结构：

```
type EstNode struct {
    children map[uint64]*EstNode
    replicas map[ReplicaID]int64 // replica → 最近见到的 unix 秒
    mu       sync.RWMutex
}
```

- 信息源仅为网关路由历史；TTL（默认 10min，随命中刷新）；Pod UID 变化/健康摆动全量摘除；Registry 自身 LRU 限内存；

- **诚实教学点**：误差只在”不同文本同 token 前缀”方向漏（保守）；但无法感知引擎内淘汰，TTL 本质是在猜。亲手做一遍才能讲透它为何被精准模式取代。

6.2 精准模式（消费引擎事件，项目技术核心）

Tokenize 问题（最关键的取舍点）：要把请求内容映射到引擎的块哈希，网关必须先拿到 token ids——这是旧版刻意回避、精准模式绕不开的成本。三个方案按序实验：

1. **调引擎 /tokenize 端点**：零新依赖，一跳额外 RTT；对 token 序列做 LRU 缓存（同 prompt 池命中率高）；先实测该跳对路由决策耗时的占比；
2. Python tokenizer sidecar（HuggingFace tokenizers）：省一跳但引入新组件与模型词汇表同步问题；
3. Go 内嵌：生态弱，不选。

默认选方案 1，数据证明一跳不可接受再升方案 2——这个决策过程本身是面试素材。

块标识（对齐 AlBrix 公开口径，现勘锁定细节）：递归哈希 $h_i = H(h_{i-1} \parallel token_i)$ ，按约 256 token/block 聚合为块哈希；前缀命中深度 = 请求块哈希链与索引匹配的最长块数 $\times$ blockSize。**事件 schema（字段名、消息类型、推送通道形态）以锁定版本的 vLLM 源码/官方文档为准，本文不虚构字段名**；现勘报告必须含一份 schema 摘录与版本号。

索引结构：

```
type PreciseIndex struct {
    block2rp map[BlockHash]map[ReplicaID]struct{} // 倒排：块 → 持有副本
    rp2blocks map[ReplicaID]map[BlockHash]struct{} // 正排：副本摘除时 O(k) 清理
    freq      *CountSketch // 块热度，核心2预热选 top-K
    mu        sync.RWMutex
}
```

- 事件处理：add/evict/全量重建三类；收到事件到索引可见的延迟单独打点（**索引滞后**是精准模式的主要误差源）；
- 失配自愈：路由到”应有前缀”的副本但引擎自报 miss（事件丢失/滞后/淘汰竞争）→ 这是性能退化不是错误；miss 率按副本监控，超过阈值触发该副本索引全量重同步；
- 路由打分仍可表达为 $score = \alpha \cdot affinity + (1-\alpha) \cdot balance$ （affinity = 命中块数归一化，balance = $1 - inflight/cap$ ）； **α 扫描降级为教学实验**，不是旗舰卖点。

6.3 旗舰图 1：精准 vs 估算 vs AlBrix 三方对照

同一负载台下，三条件（自研估算 / 自研精准 / 真实 AlBrix） $\times f \times$ 缓存压力三档，画 TTFT P50/P99 与命中率的二维曲线。结论允许任何方向——包括” $f < 0.2$ 时三者无差异，耦合引擎不值得”。**阴性边界比阳性数字更值钱**：它回答”这招什么时候没用”。

7. 核心 2：缓存感知的弹性协同（项目灵魂，默认押注候选 A）

7.1 先测量：伸缩事件研究（event study）

在自研精准路由与 AIBrix 上各做：t0 = 新副本 Ready，时间轴 -60s...+300s，画：全局 TTFT P50/P99、全局命中率、**新副本流量份额**、新副本命中率。量化两个量：

- **冷启动惩罚**：新副本达到稳态流量份额的时间（饿死判定：5 分钟份额 < 公平份额 50%）；
- **扩容收益迟滞**：从 scale 决策到全局 P99 实际改善的滞后。

缩容侧：当前常见策略（按 inflight/年龄杀）下，被杀副本独有块占比 = **缓存价值损失**，逐次记录。

7.2 设计三件套

(1) 扩容前热前缀预热

- 从 `PreciseIndex.freq` 取 top-K 热块，反推出覆盖这些块的最短前缀集合（贪心集合覆盖，K 与预算参数化）；
- 向新副本发**合成 prefill 请求**（长前缀 + `max_tokens=1`），只买 prefill 不买 decode；设预热 token 预算上限（如 KV 池 15%）与速率限制，防止预热抢在线请求的算力；
- 预热进度用块事件确认（不是假设发出即生效），未确认的块不计入新副本 readiness。

(2) 加权放量（带迟滞，不要一刀切换）

新副本就绪后不立即拿满分：

```
w(r) = clamp((readiness -  $\theta_{lo}$ ) / ( $\theta_{hi}$  -  $\theta_{lo}$ ), 0, 1) // readiness = 已确认预热块/目标块
```

w 进入软评分作为乘子； θ_{lo}/θ_{hi} 之间线性 ramp，观察窗口（默认 30s）内份额波动超阈值则冻结 ramp 防抖动。老副本 w=1。

(3) 缓存价值感知缩容 + drain

- 选副本：价值(r) = $\sum_{b \in \text{rp2blocks}[r]} \text{稀缺度}(b)$ ， $\text{稀缺度}(b) = 1 / \text{持有 } b \text{ 的副本数}$ —— 多家共有的块杀了无损，独有块价值最高；优先杀价值低者，价值并列再看 inflight；
- drain：标记停止接收新会话 → 等待在途流式请求结束（超时默认 120s，超时仅对可安全重试的请求强切，见增量池 A）→ 等待期间该副本块进入“濒死”状态，亲和打分降权但不归零；
- 防护：任何时刻可服务副本数不得低于 Little's Law 需求下限，缩容违反则中止。

7.3 何时扩：Little's Law 预测器（配角，不单独卖点）

- 到达率 EWMA ($\alpha=0.3$)； $L = \lambda \times (\text{TTFT}_{\text{预填}} + \text{out_tokens} \times \text{TPOT})$ ；需求副本数 $N = \text{ceil}(L / N_{\text{capacity}})$ ，capacity 以“KV usage 90% 时实测稳态并发”标定；
- **与冷启动故事的结合点**：预测扩容提前量 = 预热耗时（实测）+ ramp 耗时，使“副本 Ready 时热前缀也就绪”；对照组保留：CPU util HPA（开发期替代 GPU util 的错误信号，报告说明等价性论证）/ KV usage 反应式 / 本策略；
- 自研小 controller 直打 Deployment scale subresource（避开 prometheus-adapter 重依赖）；扩容立即生效、缩容冷却 5min + 两个观察窗口。

7.4 旗舰图 2：伸缩事件时间轴对比

三条件（反应式 HPA / 精准路由无协同 / 完整协同）在同一 churn 脚本下的时间轴：TTFT 尖峰高度与持续时长、命中率凹陷、扩容收益迟滞。**这是整个项目的签名图。**

7.5 终局：回馈上游

原型在自有仓库验证 → 先向 AIBrix 提 issue（贴 7.1 的测量数据与复现脚本）→ maintainer 认可方向后按其架构（Envoy 插件/Go plugin）提交 routing policy 或预热控制器 PR。**自有仓库与上游 PR 是同一条故事线的两段，不是两个项目。**

8. 增量池（只允许挑 1-2 个，超选即范围膨胀）

增量	内容要点	为什么是它
A 流式故障语义与安全重试	X-Request-Id 幂等；故障点区分：prefill 完成前挂（无副作用，安全重试）/ 完成后 mid-stream 挂（非幂等，靠客户端 last-event 续传或显式重发）；故障注入 = 随机时刻 kill 副本	成本低、区分度高、上游文档空白；与核心 2 drain 直接复用
B 多租户缓存干扰测量	租户标识透传；量化”一个租户的大批量流量驱逐另一租户前缀”；软隔离 = 按租户的块价值保护/预热配额	AIBrix 有优先级队列但无人量化缓存层面的跨租户干扰
C TTFT 归因观测	把 TTFT 拆为 排队 / prefill 重算（miss）/ KV 命中 / 网络四段	所有旗舰图的解释力底座，工程小但分析价值高

9. 调研层（只写文档/集成实验，绝不自建）

- **P/D 分离与 KV 传输**：NIXL/SHFS/Mooncake、AIBrix PriskV（单节点无 RDMA P/D）、llm-d 路线；租 GPU 时可选 2 卡跑一次官方 demo 写观感。口径永远是”调研过、理解为何需要 RDMA/高速互连”；
- **KV offload**：用 **LMCache 做一次集成实验**（CPU/SSD 分层对 TTFT 的影响），不自建 offload 层；
- **Registry HA**：行业答案已出（AIBrix = Redis 状态同步）。本项目单实例 + 周期快照为诚实范围，另写一页决策分析（为什么不从第一天上 etcd/Redis：写路径热度、读多写少、允许过期），不构建。

10. Benchmark 方法学（可复现纪律）

1. **重复次数**：每组 ≥7 次取中位数，报告 bootstrap 95% CI；只跑 3 次的数字不进报告；
2. **冷态控制**：每轮重启 vLLM 副本清空 KV；热态实验明确预热流程；轮次间进程不残留；
3. **完全锁定**：模型版本、vLLM 版本、镜像 digest、机器配置、负载配置文件全部随报告入库，`make bench COND=xxx` 一键复现；
4. **三条件同流量**：自研两模式与 AIBrix 对照组共用同一 loadgen 输出（固定随机种子的请求轨迹文件）；

5. **CPU 数据的诚实标注**：CPU 模式只对机制与相对趋势负责，绝对值无生产意义；两张旗舰图最终由 GPU 真值替换，报告保留两版并说明差异；
6. **客户端哨兵**：负载台 CPU/连接数监控曲线作为每张性能图的附件，客户端近饱和的轮次作废重跑。

11. 项目优点（面试可讲）

1. 完整走过” 现勘 → 复现 → 测量 → 发现缺口 → 设计 → 回馈上游” 的闭环，而非功能堆砌；
2. 精准/估算两种信息源亲手实现并三方对照（含真实 AIBrix），对 vLLM KV 事件机制的理解达代码级；
3. 事件研究法量化冷启动惩罚，预热-加权放量-价值感知缩容构成完整协同设计，每个参数有数据来源；
4. SSE 数据面硬货（五坑实测）、可解释排队论（Little’ s Law，不搞 ML）、严格的实验纪律；
5. 改进进入上游 issue/PR—— 第三方背书的技术判断，简历最高信用等级。

12. 诚实清单（不足主动说）

- 3 副本 kind/单机规模：机制验证，不做规模外推声明；
- CPU 后端 KV 事件支持是头号风险（§ 4），mock 事件期的一切精准模式数字标注为逻辑验证；
- 块事件 schema 跟随 vLLM 版本，升级成本真实存在（MetricsView 与事件适配层就是为此）；
- 单 Registry 无 HA（刻意范围，行业答案引用 AIBrix Redis 方案）；
- 不声称控制引擎内部淘汰；预热是” 请求级” 手段，受引擎调度影响，预热完成以事件确认而非假设；
- 核心 2 押注候选 A 有” 被上游提前解决” 的风险，闸门（§ 4）保护转向，不硬撞；
- 无 RDMA/多机 P/D 实测：调研级；GPU 真值仅覆盖 1-2 次租卡窗口。

13. 砍项清单（v2 → v3，说清为什么砍）

v2 内容	处置	理由
SLO 分级/准入/抢占（旧增量4）	砍入增量池，默认不做	AIBrix v0.7 已有 priority tier + rate limiting，重造即负分
预测式弹性作为独立卖点	降级为核心 2 的组件	单做只是” 又一个 HPA 替换品”
会话亲和（旧增量7）	删除	精准事件索引下基本是伪问题；“路由层软化” 本身说明它不该存在
Registry HA 构建	写决策分析，不构建	行业答案已产品化
α 双目标 Pareto 当旗舰图	降为核心 1 教学实验	weighted soft-scoring 已产品化，无辨识度
自建 P/D 分离、自建 KV offload	调研/LMCache 集成	多卡+高速网络门槛，上游重投入，自建必输
SSE 反代	保留在基座	唯一自研数据面的训练价值

14. 实施步骤（10 步，闸门驱动）

步骤	内容	验收 / 注意点
0 现勘	§ 4 全清单：CPU 多副本、AlBrix/ProdStack quickstart、10 PR 笔记、现勘报告、关闭 CPU 事件风险、缺口闸门定核心 2	现勘报告评审通过才进入步骤 1； 轻量部分立即并行启动，不等 P1 完工
1 基座	SSE 反代（五坑全测）+ MetricsView + 负载台 + Grafana	流式对话稳定、杀后端不挂连接；负载台三种策略可切换
2 基线报告	RR vs least-outgoing，负载台工作区标定	客户端不饱和证据随报告
3 估算索引	§ 6.1 radix + TTL + 摘除	机制正确，文档标注”被淘汰路线复现”
4 精准索引	tokenize 方案选型实测 + 事件消费 + 双倒排 + 失配重同步	索引滞后有打点；事件 schema 锁版本
5 旗舰图1	三条件 × f × 缓存压力对照（含真实 AlBrix）	二维曲线 + 95% CI；负边界写清
6 冷启动测量	§ 7.1 事件研究（自研 + AlBrix 各一轮）	饿死/迟滞/价值损失三组量化 ← issue 素材
7 协同设计	预热 + 加权放量 + 价值感知缩容 + drain	旗舰图2：尖峰/凹陷/迟滞三条件对比
8 增量池	只挑 1 个（默认 A：流式故障语义）	独立小报告
9 真值校准	租 GPU（Qwen 7B 级）重跑旗舰图 1/2；可选 LMCache 与 2 卡 P/D demo	真数据替换 CPU 数据，双版保留
10 回馈	issue（数据）→ 讨论 → PR（按 AlBrix 架构）	≥1 代码级 PR；被拒也保留”为什么”的完整故事

拼接逻辑：1+2 可测基座；3+4+5 = 双模式复现（第一个完整故事）；6+7 = 前沿纵深（第二个故事 + PR）；9 收口真值；10 把信用落到上游。任意层可停笔写简历，但秋招包目标是 1-7 + 10。

15. 排期与死线（倒推）

- **立即（P1 期间并行）：**步骤 0 的轻量部分（vLLM CPU + AlBrix quickstart + 读码），每周 ≤0.5 天，不抢 P1 主线；
- P1 收尾后全职 P2：步骤 1-5 约 9-11 周，6-7 约 6-8 周，8 约 2-3 周，9 约 1 周；
- **2027.8 秋招包最低构成：**基座 + 核心 1（含旗舰图1）+ 核心 2 主实验（旗舰图2 初稿）+ ≥1 个 AlBrix PR（docs/test PR 不计入，代码级；若 PR 流程未闭合，至少 issue 有 maintainer 正式回应）；
- 砍项顺序（时间不够时）：增量池 → GPU 校准的 demo 类部分（旗舰图校准不可砍）→ 核心 2 的 drain 细化；核心 1 与旗舰图 1 不砍。

16. 预期成果

交付物：KVRouter 仓库（代码 + README + 设计决策记录 ADR + 复现命令）、负载台（独立可用，可打 AlBrix）、现勘报告、两份旗舰图实验报告、博客 2 篇（SSE 反代五坑 / 精准 vs 估算复现笔记）、AlBrix issue 与 PR 链接。

简历条目草稿（V1 主版）：

独立复现并量化 vLLM 前缀路由的两种信息源机制（引擎 KV 事件精准模式 vs 路由历史估算模式），同负载台与真实 AlBrix 三方对照，给出收益随共享前缀比例/缓存欠配度变化的边界曲线；定位弹性伸缩期 KV 冷启动惩罚（事件研究法量化冷副本饿死与扩容迟滞），设计热前缀预热-加权放量-缓存价值感知缩容协同策略，伸缩期 TTFT 尖峰 $\downarrow X\%$ 、收益迟滞 $\downarrow Y\%$ ；测量与改进以 N 个 issue/PR 回馈 AlBrix (≥ 1 代码级 merged)。